

COS30018 - Option C - Task 1 Report

Stock Prediction System Setup and Evaluation

Student Name: Kha Anh Nguyen
Student ID: 103814796
Date: August 17, 2025
Course: COS30018 - Intelligent Systems
Project: FinTech101 Stock Price Prediction System

Table of Contents

COS30018 - Option C - Task 1 Report 1

 Stock Prediction System Setup and Evaluation..... 1

 1. Environment Setup 1

 2. Code Testing Results 3

 3. Understanding of v0.1 Code Base..... 6

 4. Performance Comparison 7

 5. GitHub Repository Setup 8

 Appendix 8

1. Environment Setup

1.1 Virtual Environment Configuration

A unified virtual environment was created to support both v0.1 and P1 implementations, ensuring compatibility and dependency management.

Setup Process:

```
# Create and activate virtual environment
python -m venv venv
venv\Scripts\activate # Windows
source venv/bin/activate # Linux/Mac

# Install dependencies
pip install -r requirements.txt
```

1.2 Requirements File Development

The requirements.txt file was developed to address dependencies for both projects:

```
# Essential packages for stock prediction project
# Core machine learning
tensorflow>=2.19.0
keras>=3.11.1
scikit-learn>=1.7.1
numpy>=2.1.3
pandas>=2.3.1

# Data visualization
matplotlib>=3.10.5

# Stock data fetching (we use yfinance, keeping yahoo-fin for compatibility)
yfinance>=0.2.65
yahoo-fin>=0.8.9.1
pandas-datareader>=0.10.0

# HTTP requests and web scraping
requests>=2.32.4
requests-html>=0.10.0
beautifulsoup4>=4.13.4
lxml>=6.0.0
lxml_html_clean>=0.4.2

# Jupyter notebook support (optional)
jupyter>=1.0.0
ipykernel>=6.30.1

# Utilities
python-dateutil>=2.9.0.post0
pytz>=2025.2
loguru>=0.7.0
```

1.3 Installation and Compatibility Issues

Packages Issues Resolved:

1. **yfinance vs yahoo_fin**: Replaced unreliable yahoo_fin with stable yfinance library
2. **TensorFlow Compatibility**: Fixed deprecated parameter (huber-loss -> huber, input to Input() layer) usage in P1
3. **lxml Dependencies**: Resolved HTML parsing compatibility issues

2. Code Testing Results

2.1 v0.1 Testing Results

Code Additions:

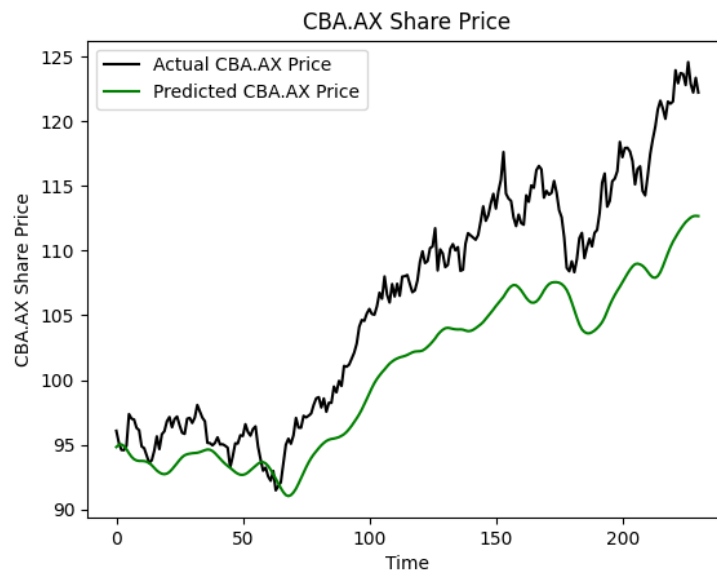
- Csv file save for metrics.

Output Files Generated:

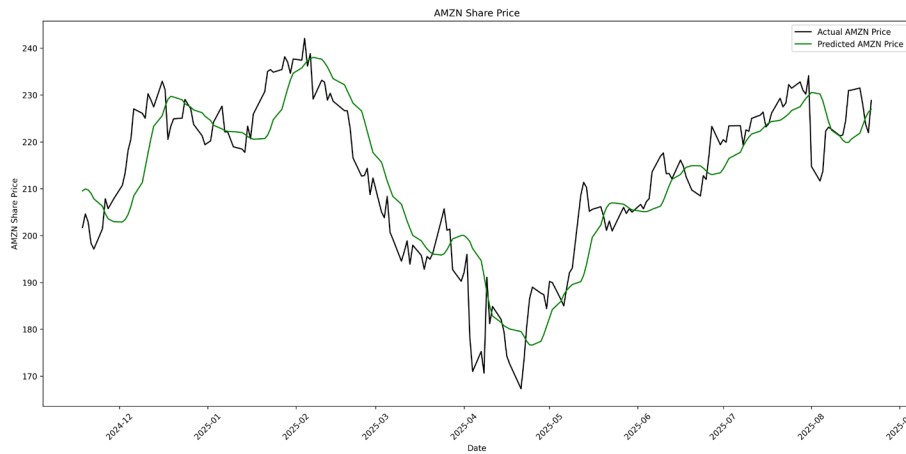
- CSV results: CBA.AX_next_day_20250817_205649.csv

```
Future price after 1 day is 224.91$  
mean_squared_error loss: 0.0063705481588840485  
Mean Absolute Error: 5.0741383622332314  
Accuracy score: 0.5365853658536586  
Total buy profit: 24.769989013671875  
Total sell profit: 7.9199981689453125  
Total profit: 32.68998718261719  
Profit per trade: 0.7973167605516387
```

- Performance plot: CBA.AX_plot.png



AMZ (for easier comparison with P1)



2.2 P1 Testing Results

Fixes Applied:

Fixed data loading function using yfinance

```
def get_stock_data(ticker):  
    """  
    Get stock data using yfinance (modern version of yahoo_fin).  
    """  
    # Default to 2 years of data  
    end_date = datetime.now()  
    start_date = end_date - timedelta(days=730)  
    data = yf.download(ticker, start=start_date, end=end_date, progress=False)  
  
    if data is not None and not data.empty:  
        # Handle multi-level columns from yfinance  
        if isinstance(data.columns, pd.MultiIndex):  
            # Flatten the column names  
            # yfinance sometimes returns data with multi-level col names like:  
            # ('Close', 'AAPL')  
            # ('Volume', 'AAPL')  
            # ('High', 'AAPL')  
            # => We want to flatten them to: ['close', 'volume', 'open', 'high', 'low']  
            data.columns = [col[0].lower() if col[1] == ticker else f"{col[0].lower()}_{col[1].lower()}"]  
        for col in data.columns:  
            # Rename 'close' to 'adjclose' to match expected format  
            if 'close' in data.columns:  
                data = data.rename(columns={'close': 'adjclose'})  
        print(f"Successfully loaded {ticker} data using yfinance")  
    return data
```

```
else:  
    raise ValueError(f"Failed to load data for {ticker}")
```

Fixed loss function compatibility

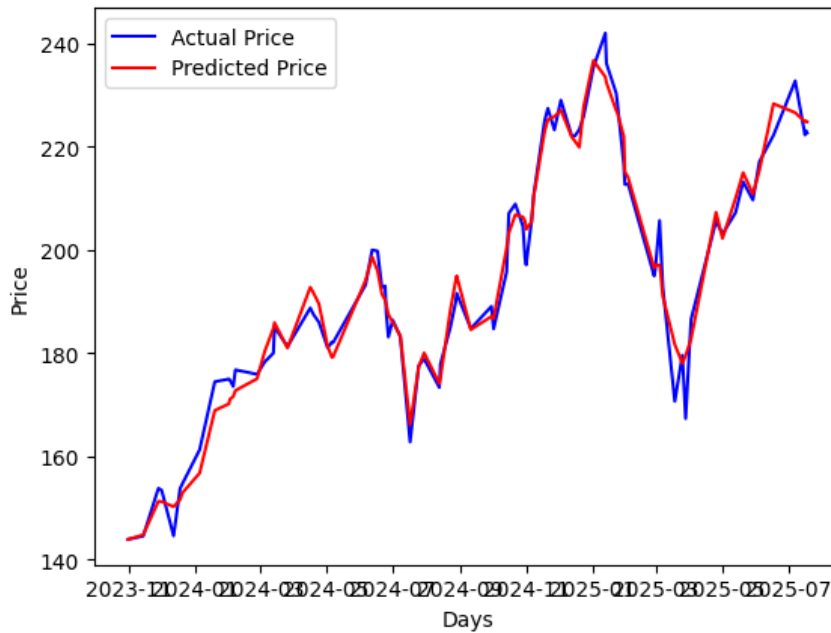
```
LOSS = "huber" # change from "huber_loss"
```

Performance Results:

```
Future price after 15 days is 223.07$  
huber: 0.00044861173955723643  
Mean Absolute Error: 2.7664683948863638  
Accuracy score: 0.9545454545454546  
Total buy profit: 559.6899871826172  
Total sell profit: 355.78997802734375  
Total profit: 915.4799652099609  
Profit per trade: 10.403181422840465
```

Output Files Generated:

- Model weights: Saved to p1/results/
- **CSV results:** p1_output.csv
- Performance visualization:



3. Understanding of v0.1 Code Base

3.1 Architecture Overview

The v0.1 implementation represents a basic LSTM-based stock prediction system with the following characteristics:

Core Components:

1. **Data Loading:** Yahoo Finance via yfinance library
2. **Preprocessing:** MinMaxScaler normalization (0-1 range)
3. **Sequence Creation:** 60-day sliding windows
4. **Model Architecture:** Stacked LSTM with dropout layers
5. **Prediction:** Single next-day price forecast

Model Structure:

```
model = Sequential([
    LSTM(units=50, return_sequences=True, input_shape=(60, 1)),
    Dropout(0.2),
    LSTM(units=50, return_sequences=True),
    Dropout(0.2),
    LSTM(units=50),
    Dropout(0.2),
    Dense(units=1)
])
```

3.2 Algorithm Flow

Step-by-Step Process:

1. **Training Data Collection:** Downloads historical stock data (2020-2023)
2. **Training Data Normalization:** Scales ONLY training data closing prices to [0,1] range using MinMaxScaler fitted on training data
3. **Sequence Generation:** Creates 60-day input sequences from training data to predict next day
4. **Model Training:** Trains LSTM network for 25 epochs with batch size 32
5. **Test Data Processing:** Downloads separate test period data (2023-2024)
6. **Test Data Scaling:** Applies same scaler (fitted on training data) to test data
7. **Prediction:** Generates predictions and inverse transforms using same training scaler.

4. Performance Comparison

4.1 v0.1 and P1 comparison

Aspect	v0.1 Approach	P1 Approach	Analysis
Stock Symbol	AMZN (Amazon Inc.)		Identical for fair comparison
Data Period	Last 730 days (dynamic)		Identical timeframe
Train/Test Split	Sequentially, load train and test data separately	Randomly (configurable: 80%/20%)	v0.1 is more manual
Scaling Strategy	Training data only (prevents test leakage)	All data together (test value range leakage)	v0.1 theoretically correct
Prediction Horizon	1 day ahead	15 days ahead	P1 has longer predictions horizon
Features Used	1 (Close price only)	5 (Open, High, Low, Close, Volume)	P1 has 5x more information
Model Architecture	Simple LSTM (3 layers, 50 units)	More complex LSTM (2 layers, 256 units)	P1 more sophisticated
Training Epochs	25	500	P1 trains 20x longer
Training Loss	0.00637 (mse)	0.00044 (huber)	P1 achieves 14x lower training loss
Mean Absolute Error	5.074 %	2.766 %	P1 has lower MAE
Accuracy Score	53.7 %	95.5 %	P1 has 78% improvement
Total Buy Profit	\$24.77	\$559.69	P1 is 23x better in buy profit
Total Sell Profit	\$7.92	\$355.79	P1 is 44x better in sell profit
Total Profit	\$32.69	\$915.48	P1 is 28x more profitable
Profit per Trade	\$0.797	\$10.403	P1 has 13x increase in profit for each trade

4.2 Comparison breakdown

Visual Analysis:

- **v0.1** has:
 - Shows prediction delays when following market trends.
 - Smoother predictions, which miss out critical peaks and valleys.
 - Underestimates peaks during upward trends
 - Overestimates peaks during downward trends.
 - Less responsive behavior to market volatility.
- **p1**:
 - More responsive prediction to the market trend
 - Follow the trend better but still miss some peaks.

Note: Visual comparison has limitations due to different splitting techniques: P1 has random train/test split while v0.1 has sequential.

P1 has **better prediction accuracy** (95.5% vs 53.7%) and **trading profitability** (\$915.48 vs \$32.69 total profit). However, **v0.1** offers **advantages in computational efficiency** with 20x faster training (25 vs 500 epochs) and theoretically sound data handling that prevents information leakage. The trade-off between accuracy and speed makes P1 optimal for production trading systems, while v0.1 suits rapid prototyping and resource-constrained environments.

5. GitHub Repository Setup

Highlight structure for the Github Repository:

```
stock-prediction /
├── v0.1/                # Original YouTube tutorial code (fixed)
│   ├── v0.1.py          # Fixed stock prediction implementation
│   └── results/         # Output files and results
├── p1/                 # GitHub project implementation
│   ├── p1.py            # Core functions and data processing
│   ├── train.py         # Model training script
│   ├── test.py          # Model evaluation script
│   ├── parameters.py    # Configuration parameters
│   ├── p1.ipynb         # Jupyter notebook version
│   └── results/         # Model outputs and results
├── dev/                # For task 2
├── utils/              # Shared utilities (evaluating_data, plots)
├── requirements.txt     # Package dependencies
└── README.md           # This file
```

Appendix

Files run successfully

V0.1:

```

● (venv) (base) PS C:\Users\User\OneDrive - Swinburne University\IS\assignment1\code> python v0.1\v0.1.py
2025-08-17 20:56:02.797967: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=
0'.
2025-08-17 20:56:04.851410: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results
due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=
0'.
C:\Users\User\OneDrive - Swinburne University\IS\assignment1\code\v0.1\v0.1.py:51: FutureWarning: YF.download() has changed argument auto_adjust
default to True
data = yf.download(COMPANY, TRAIN_START, TRAIN_END)
[*****100%*****] 1 of 1 completed
2025-08-17 20:56:07.318 | INFO | __main__:<module>:52 -
data
2025-08-17 20:56:07.321 | INFO | __main__:<module>:68 -
2D scaled data
2025-08-17 20:56:07.321 | INFO | __main__:<module>:92 -
3D scaled data
2025-08-17 20:56:07.323 | INFO | __main__:<module>:102 -
x_train
2025-08-17 20:56:07.323 | INFO | __main__:<module>:103 -
y_train
2025-08-17 20:56:07.323 | INFO | __main__:<module>:108 -
x_train
2025-08-17 20:56:07.316478: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instruc
tions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flag
s.
C:\Users\User\OneDrive - Swinburne University\IS\assignment1\code\venv\Lib\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pas
s an 'input_shape' / 'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the
model instead.
super().__init__(**kwargs)
Epoch 1/25
27/27 ----- 3s 28ms/step - loss: 0.0659
Epoch 2/25
27/27 ----- 1s 26ms/step - loss: 0.0102
Epoch 3/25
27/27 ----- 1s 25ms/step - loss: 0.0078
Epoch 4/25
27/27 ----- 1s 26ms/step - loss: 0.0078
Epoch 5/25
27/27 ----- 1s 25ms/step - loss: 0.0067

```

```

Epoch 6/25
27/27 ----- 1s 29ms/step - loss: 0.0055
Epoch 22/25
27/27 ----- 1s 27ms/step - loss: 0.0049
Epoch 23/25
27/27 ----- 1s 26ms/step - loss: 0.0051
Epoch 24/25
27/27 ----- 1s 26ms/step - loss: 0.0052
Epoch 25/25
27/27 ----- 1s 27ms/step - loss: 0.0049
C:\Users\User\OneDrive - Swinburne University\IS\assignment1\code\v0.1\v0.1.py:198: FutureWarning: YF.download() has changed argument auto_adjust
default to True
test_data = yf.download(COMPANY, TEST_START, TEST_END)
[*****100%*****] 1 of 1 completed
2025-08-17 20:56:28.651 | INFO | __main__:<module>:200 -
test_data
2025-08-17 20:56:28.651 | INFO | __main__:<module>:208 -
actual_prices
2025-08-17 20:56:28.651 | INFO | __main__:<module>:211 -
total_dataset
2025-08-17 20:56:28.651 | INFO | __main__:<module>:217 -
model_inputs
2025-08-17 20:56:28.651 | INFO | __main__:<module>:220 -
model_inputs reshaped
2025-08-17 20:56:28.651 | INFO | __main__:<module>:246 -
x_test
8/8 ----- 0s 37ms/step
1/1 ----- 0s 30ms/step
2025-08-17 20:56:49.715 | INFO | __main__:<module>:283 - Prediction: [[115.16059]]

Next Day Prediction: $115.16
Last Known Price: $122.22
Predicted Change: $-7.06
Error: 5.8%
CSV saved: v0.1/results/CBA_AX_next_day_20250817_205649.csv

```

P1:

